

AI 데이터 분석 핵심 라이브러리

Pandas 기초: 입문과 데이터 불러오기

지하철 공기압축기(MetroPT-3) 실데이터 기반 표 데이터 분석 입문 가이드

AI 데이터 분석 교재 | 대상: 데이터 분석 학습자

1. 코드로 데이터를 다루는 이유 (Pandas vs 엑셀)

PART 01. Pandas 입문

설비 데이터의 규모적 특성

- 1초 간격 수집 센서 1대: 하루 **86,400줄** 누적
- 수십 개 센서 모니터링: 하루 수백만 줄 돌파
- 엑셀의 한계: 최대 1,048,576행 (수십만 줄만 넘어도 버벅임)

코드로 다룰 때의 3대 이점

1. **대량 처리**: 수백만 줄도 한 줄 명령어로 자동 계산
2. **재사용성**: 한 번 작성한 코드는 새 데이터에 재실행 가능
3. **정확성**: 클릭 실수 없이 동일한 작업 정밀 반복

엑셀과 Pandas의 역할 구분

- **엑셀**: 적은 양의 데이터를 눈으로 직접 보며 서식/색칠할 때 적합
- **Pandas**: 대용량 반복 데이터 처리 및 대규모 자동화에 최적화
- **실무 표준**: 무거운 연산은 Pandas로 완료하고, 최종 결과만 엑셀로 전달!

2. Pandas의 핵심 구조: Series와 DataFrame

PART 01. DataFrame 구조

Series (1차원 한 줄 데이터)

- 표의 **열 하나(Column)**에 해당하는 조각
- 동일한 의미의 값들이 인덱스 번호표와 함께 순서대로 모인 형태
- `df[ '오일온도' ]` 추출 결과는 바로 Series가 됨

DataFrame (2차원 표 전체 그릇)

- 여러 개의 Series가 옆으로 결합된 **2차원 표 전체**
- 행(Row), 열(Column), 인덱스(Index)의 3요소로 구성
- CSV 파일을 불러오면 기본적으로 DataFrame 객체 생성

행(Row) · 열(Column) · 인덱스(Index) 정의

- **Row (행):** 특정 시점의 1건 측정 기록 (가로 한 줄)
- **Column (열):** 하나의 측정 항목 / 변수 (세로 한 칸)
- **Index (인덱스):** 각 행의 왼쪽 번호표 (0부터 시작하는 RangeIndex)

3. Pandas 데이터 분석 4단계 기본 작업 흐름

PART 01. 작업 흐름

단계	주요 동작	대표 함수 및 명령	비유적 의미
STEP 01	불러오기	<code>pd.read_csv('file.csv')</code>	분석의 관문 (현관문 열기)
STEP 02	구조 확인	<code>df.shape, df.info()</code>	데이터의 뼈대 둘러보기 (방 구조 확인)
STEP 03	미리보기	<code>df.head(), df.tail()</code>	실제 데이터를 눈으로 확인 (방 들여다보기)
STEP 04	통계 요약	<code>df.describe()</code>	대표값 및 이상 신호 포착 (평균 및 상태 파악)

4. CSV 파일의 이해와 도구별 차이

PART 02. CSV 이해

CSV (Comma-Separated Values) 란?

- 각 항목의 값을 **쉼표(,)**로 구분하여 저장한 단순 텍스트 파일
- 맨 윗줄은 열 이름인 **헤더(Header)**, 그 아래는 데이터 행
- 메모장, 엑셀, 파이썬 등 모든 시스템에서 호환되는 표준 포맷

열린 도구에 따른 시각적 차이

- **메모장**: 2020-02-27,9.3,51.3,가동 (쉼표가 그대로 노출)
- **엑셀**: 쉼표를 기준으로 칸을 나눠 깔끔한 표로 시각화
- **Pandas**: 쉼표를 구분해 코드로 조작 가능한 DataFrame으로 변환

5. NumPy/Pandas 불러오기 및 read_csv 기본

PART 02. read_csv

불러오기 관례: `import pandas as pd`

- 전 세계 데이터 분석가들의 공통 약속 (별칭 `pd`)
- 노트북 맨 위 첫 번째 셀에 한 번만 실행하면 계속 유지

```
import pandas as pd
import numpy as np
```

변수 저장의 필연성

- `pd.read_csv(...)`만 실행하면 화면에 잠깐 찍히고 데이터가 사라짐
- 반드시 **`df = pd.read_csv(...)`**와 같이 변수에 할당해야 지속 사용 가능!

```
# CSV 읽어 변수 df에 담기
df = pd.read_csv("data/12_metro_compressor.csv")
df.head()
```

6. 파일 경로와 FileNotFoundError 진단

PART 02. 경로

| 상대 경로 vs 절대 경로

- **상대 경로:** 현재 작업 중인 코드 위치 기준 (예: data/12_metro.csv)
- **절대 경로:** C 드라이브 루트부터 전체 주소 지정
- **경로 구분자:** 윈도우 역슬래시(`\`) 대신 슬래시(/) 사용 권장

| FileNotFoundError 점검 3단계

1. **철자/대소문자:** `compressor` ↔ `compresser` 오타 확인
2. **확장자:** `.csv` 확장자가 누락되었는지 확인
3. **폴더 위치:** 현재 위치와 `data/` 하위 폴더 여부 확인

7. read_csv 핵심 옵션 (encoding, sep, usecols, nrows)

PART 02. 옵션

옵션명	설명 및 사용 상황	예제 구문
encoding	한글 깨짐 해결 (기본 `utf-8-sig`, 윈도우 엑셀 `cp949`)	<code>pd.read_csv('data.csv', encoding='utf-8-sig')</code>
sep	구분자가 쉼표가 아닐 때 지정 (세미콜론 `;`, 탭 `\t` 등)	<code>pd.read_csv('data.csv', sep=';')</code>
usecols	열이 많을 때 필요한 특정 열만 골라 로드 (메모리 절약)	<code>pd.read_csv('data.csv', usecols=['측정시각', '오일 온도'])</code>
nrows	수백만 줄 중 위에서 일부 행만 빠르게 가져와 구조 확인	<code>pd.read_csv('data.csv', nrows=10)</code>

8. [실습 활용 예제] read_csv 다양한 옵션 적용

실습 가이드 참고

시나리오: 세미콜론 구분자 및 한글 인코딩이 적용된 지하철 공기압축기 데이터 읽기

```
import pandas as pd

# 1. 구분자가 세미콜론(;)인 파일 읽기 (sep 옵션)
df_semi = pd.read_csv("data/12_metro_compressor_semicolon.csv", sep=";")
print("세미콜론 적용 shape:", df_semi.shape) # (200, 7)

# 2. 한글 인코딩 및 필요한 열 3개만 지정하여 불러오기 (encoding, usecols)
df_selected = pd.read_csv(
    "data/12_metro_compressor_semicolon.csv",
    sep=";",
    encoding="utf-8-sig",
    usecols=["측정시각", "오일온도", "모터전류"]
)
print("열 선택 로드 shape:", df_selected.shape) # (200, 3)
print(df_selected.head(3))
```

9. 데이터 불러오기 문제 해결 점검 루틴

SUMMARY

1. 경로 점검

FileNotFoundError

철자, `.csv` 확장자,
`data/` 경로 위치 재확인

2. 인코딩 점검

한글 글자 깨짐

`utf-8-sig` 시도 후 안 되
면 `cp949` 적용

3. 구분자 점검

한 열로 값 뭉침

메모장으로 원본 확인 후
`sep=";"` 지정

4. 정상 확인

최종 확인

`df.head()`를 찍어 값이 잘
끔하게 나뉘었는지 검증

★ 다음 단원: DataFrame 구조 파악 및 통계 미리보기 마스터하기!